WINIT

NY Parking Ticket Tracker

Fullstack & AI Technical Challenge

Context

Your goal is to build a professional-grade tool for fleet management companies to monitor parking summons in real-time. The application must process a CSV of ticket IDs, scrape the New York DMV website asynchronously, and stream progress updates to a modern dashboard.

Required Stack

- Frontend: React + Tailwind
- Backend: Python (FastAPI)
- Comm: Server-Sent Events (SSE)
- Scraping: Playwright/Selenium
- AI (Optional): Claude API

Key Features

1. Bulk Import & Validation

2. Async Scraping Engine

- Users upload a CSV file with: `ticket_id`, `plate_number`, `state_code`.
- The backend must validate the CSV structure and handle malformed data gracefully.

- **Target Website:** https://process.dmv.ny.gov/WebSummons/ **(REQUIRED)**
- **Navigation Flow:** Navigate through "No, I do not have a NY State license" flow and query each ticket.
- **Data Extraction:** Parse ALL ticket data fields as shown in the attached PDF reference.
- **Non-blocking:** The scraping process must run in the background.
- **SSE Streaming:** Emit live updates (e.g., "Searching Ticket #102...", "Found - $65 fine", "Scraping complete").

3. Claude Agents (Optional)

- **Data Structuring:** Convert messy HTML into clean JSON.
- **Actionable Insights:** Analyze if a ticket is eligible for dispute.
- *Explain why an Agentic approach was chosen in README.*

HIGH PRIORITY

4. Professional UX/UI

This needs to be beautiful. We are looking for a premium user experience.

Dashboard: A clean interface featuring a global progress bar and a "Live Feed" of scraper activities.

Results Table: A polished, sortable list showing ticket status, amounts, and dates.

Empty States: gracefully handle cases where no tickets are found or the DMV site is down.

Visual Polish: Use modern shadows, clean typography, and consistent spacing.

Deployment (Bonus Points)

A major bonus will be awarded if the application is deployed and accessible via a public URL:

- **Frontend:** Vercel / Netlify.
- **Backend:** DigitalOcean, Render, or Railway.
- **Proxying:** Ensure the SSE connection works correctly through the production proxy/load balancer.

Evaluation Criteria

- **Code Quality:** Clean architecture, separation of concerns, and robust error handling.
- **Concurrency:** Ability to handle multiple scraping tasks without hanging the server.
- **Documentation:** README.md explaining how to run (Docker preferred) and deployment strategy.
- **Resilience:** Handling rate-limiting or CAPTCHAs (Mocking allowed if blocked, provided SSE logic is sound).

Deliverables

- A link to the GitHub/GitLab repository.
- A live URL of the deployed application (if applicable).
- A brief summary of architecture choices regarding SSE and AI Agents.

Attachments included with this challenge:

📄 **.CSV file** - Sample ticket data for testing
📄 **.PDF file** - Reference showing all ticket data fields to parse from the DMV website

© WINIT Technical Recruitment